Pointer-First Agent Constitution and Operational Certification of the Stela Repository

1. Executive Summary: The "Total Explainability" Mandate

The Stela repository operates under a "Pointer-First" constitution designed to eliminate "ghost canon" and enforce a "Flat Truth" topology. However, a strict audit against the "Total Explainability" standard reveals a critical asymmetry between the **Explain** layer (Agents, Tasks, Skills) and the **Run** layer (Tools, Linters, Scripts).

While the user-facing binaries (ops/bin/) are specified, the internal logic of the system (tools/, ops/lib/) operates as "Shadow Logic"—functional but undocumented code. To achieve the required certification, the system must eradicate this shadow logic. **Every executable file must be registered in a bureaucratic ledger AND described in a narrative specification.**

Current Certification Status: NON-COMPLIANT

- **Functional Integrity: PASS** (Code works and enforces PoT).
- **Registry Compliance: FAIL** (5 missing registries).
- **Specification Compliance: FAIL** (Missing specifications for 15+ internal scripts).

This report details the specific gaps and provides the mandatory action plan to bring the repository into full compliance.

2. Constitutional Architecture and Governance Topology

The certification is grounded in the repository's tripartite governance structure:

2.1 The Policy of Truth (PoT): The Constitutional Core

PoT.md is the Single Source of Truth.[1] It defines the "Filing Doctrine" (ops/=Run, docs/=Explain). The "Total Explainability" mandate extends this doctrine: if code exists in ops/ or tools/, it must have a corresponding "Explain" artifact in docs/. Currently, this mirror is broken for the toolchain.

2.2 The State of Play (SoP): The Historical Ledger

SoP.md records the evolution of the system.[1] Recent entries (DP-OPS-0056) show a trend

toward hardening the Task subsystem. Certification requires that the *creation* of these missing specs and registries be logged as a governance hardening event in the SoP.

2.3 The Task Dashboard (TASK): The Active Interface

TASK.md manages the active workload.[1] It relies on tools/lint/dp.sh and tools/lint/task.sh to enforce its schema. These linters are currently "black boxes" to the documentation—they enforce rules that are not explicitly documented in a spec file, violating the "Explainable" requirement.

3. Operational Kernel Certification (ops/bin)

The ops/bin directory contains the operator-facing executables.

Binary	**Spec File (docs/ops/specs/binaries/)**	**Registry (BINARIES.md)**	**Verdict**
context	context.md (Exists)	**MISSING**	**Incomplete**
dump	dump.md (Exists)	**MISSING**	**Incomplete**
help	help.md (Exists)	**MISSING**	**Incomplete**
llms	llms.md (Exists)	**MISSING**	**Incomplete**
map	map.md (Exists)	**MISSING**	**Incomplete**
open	open.md (Exists)	**MISSING**	**Incomplete**
project	project.md (Exists)	**MISSING**	**Incomplete**
prune	prune.md (Exists)	**MISSING**	**Incomplete**

Gap Analysis:

- **Specification Status: 100% Covered.** The technical specs exist.[1]
- **Registry Status: 0% Covered.** docs/ops/registry/BINARIES.md does not exist.
- **Action:** Create the registry to link the binary to its spec.

4. Immune System Certification (tools/)

This layer enforces governance but currently violates the "Explainability" standard by lacking documentation.

4.1 Linting Suite (tools/lint/)

Script	Spec File (docs/ops/specs/tools/lint/)	Registry (LINT.md)	Verdict
agent.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
context.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
dp.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
library.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
llms.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
project.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
style.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
task.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
truth.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**

Gap Analysis:

- **Directory Missing:** docs/ops/specs/tools/lint/ does not exist.
- **Registry Missing:** docs/ops/registry/LINT.md does not exist.
- **Impact:** The rules enforced by these linters (e.g., "No contractions" in style.sh) exist only in code. They must be elevated to documentation (docs/ops/specs/tools/lint/style.md).

4.2 Testing & Verification (tools/test/ & tools/)

Script	Spec File	Registry (TEST.md	Verdict

	(docs/ops/specs/...)	/ TOOLS.md)	
tools/test/agent.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
tools/verify.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**

Action:

- Create docs/ops/specs/tools/test/agent.md describing the agent integrity checks.
- Create docs/ops/specs/tools/verify.md describing the repository skeleton verification logic.

5. Logic Subsystem Certification (ops/lib/scripts)

These scripts provide the backend logic for the platform.

Script	Spec File (docs/ops/specs/scripts/)	Registry (SCRIPTS.md)	Verdict
agent.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
heuristics.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
project.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
skill.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**
task.sh	**MISSING**	**MISSING**	**NON-COMPLIANT**

Gap Analysis:

- **Directory Missing:** docs/ops/specs/scripts/ does not exist.
- **Registry Missing:** docs/ops/registry/SCRIPTS.md does not exist.
- **Impact:** The "Harvest/Promote" logic used to evolve the system is undocumented.

6. Library Certification (Agents, Tasks, Skills)

This layer is the **Gold Standard** for the repository. It effectively demonstrates the target state

for the rest of the system.

- **Agents:** Registered (AGENTS.md) and Specified (docs/library/agents/*.md). **CERTIFIED.**
- **Tasks:** Registered (TASKS.md) and Specified (docs/library/tasks/*.md). **CERTIFIED.**
- **Skills:** Registered (SKILLS.md) and Specified (docs/library/skills/*.md). **CERTIFIED.**

7. Required Actions for Full Certification

To achieve "Total Explainability," the following artifacts must be created. This is not optional; it is a requirement for the system to be considered "Explainable."

Phase 1: Bureaucratic Registration (The Registries)

Create the following files in docs/ops/registry/. Each file must list every file in its domain, its purpose, and a link to its specification.

1. **BINARIES.md**: Register all 8 binaries in ops/bin/.
2. **LINT.md**: Register all 9 linters in tools/lint/.
3. **TEST.md**: Register tools/test/agent.sh.
4. **TOOLS.md**: Register tools/verify.sh.
5. **SCRIPTS.md**: Register all 5 scripts in ops/lib/scripts/.

Phase 2: Technical Specification (The Specs)

Create the missing specification directories and populate them with markdown files describing the **Logic**, **Inputs**, **Outputs**, and **Failure States** of each tool.

A. Create Directories:

- docs/ops/specs/tools/
- docs/ops/specs/tools/lint/
- docs/ops/specs/tools/test/
- docs/ops/specs/scripts/

B. Draft Specifications:

- **For tools/lint/*.sh**: Create 9 spec files (e.g., style.md explaining *exactly* which regex patterns are forbidden).
- **For tools/verify.sh**: Create verify.md defining the mandatory directory structure.
- **For ops/lib/scripts/*.sh**: Create 5 spec files explaining the harvest/promote algorithms and heuristic logic.

8. Final Verdict

The Stela repository is currently **Structurally Sound** but **Bureaucratically Deficient**.

The "Run" layer (ops/bin, tools/) is operating as "Ghost Code"—it functions correctly but lacks

the registered, specified proof of existence required by the strict "Total Explainability" standard.

Certification: WITHHELD until the 5 missing registries and 16 missing specification documents are created.

Works cited

1. dump-full-main-d760e443.txt